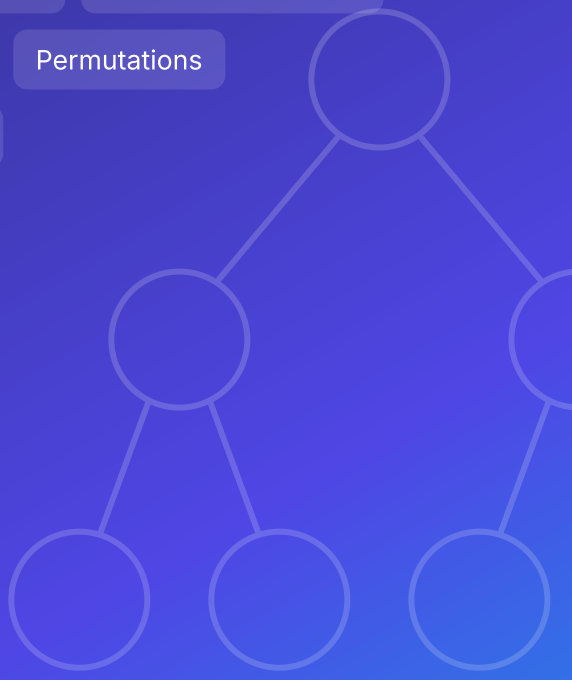
HELLO DSA · MAYANK KI VIDEO SERIES

DSA Notes Roman Urdu mein

Recursion se le kar Time/Space Complexity aur Linked List tak — asaan zaban, misalen aur diagrams ke saath

DSA kya hai | Recursion & PMI | Fibonacci tree | Head vs Tail | Arrays par recursion | Binary Search | Merge Sort | Quick Sort | Subsequence | Permutations | Tower of Hanoi | Big-O | Space Complexity | Linked List



Source: video transcripts + github.com/mayank953/HelloDSA handbook · Code Python mein hai · Ye notes revision ke liye banaye gaye hain

Fehrist (Contents)

01	DSA kya hai aur kyun zaroori hai	Data structure + Algorithm
02	Recursion ki bunyaad	Call stack, base case, limit
03	PMI — Recursion ka recipe	3 steps ka formula
04	Fibonacci, Recursion Tree, Head vs Tail	Calls ka order
05	Arrays par Recursion	Sum, sorted, indices
06	Searching & Sorting	Binary search, merge, quick
07	Strings, Subsequence & Permutations	Choices tree
08	Keypad Combinations & Tower of Hanoi	Classic problems
09	Time Complexity	Big-O, Ω, Θ, recurrences
10	Space Complexity	Auxiliary space, recursion stack
11	Linked List	Node, insert, delete
12	Quick Revision Cheat-Sheet	Exam se pehle

IN NOTES KO KAISE PARHEIN

Har chapter mein rangon wale boxes hain:

- **Yaad Rakho** — sab se aham points, exam/interview ke liye.
- **Misal (Analogy)** — rozmarra zindagi ki misal taake concept dimagh mein baith jaye.
- **Tip** — coding ya debugging ka faida mand tareeqa.
- **Galti Mat Karna** — woh ghaltiyan jo aksar students karte hain.

Code khud chala kar dekho, tod kar dekho — tabhi samajh aata hai.

CHAPTER 01

DSA kya hai?

Data Structure + Algorithm — data ko rakhne ka tareeqa aur us par kaam karne ka tareeqa

DSA do cheezon ka naam hai: **Data Structure** aur **Algorithm**.

DATA STRUCTURE

Data ko **store aur organize** karne ka tareeqa — taake usay jaldi dhoonda, badla ya nikala ja sake. Jaise: array, list, stack, queue, tree, graph, hashmap.

ALGORITHM

Kisi masle ko hal karne ke **wazeh steps** — ek recipe ki tarah. Jaise: search karna, sort karna, rasta dhoondna.

MISAL: KITABON KI SHELF

Agar 1000 kitaben ek dher mein pari hon to ek kitab dhoondne mein ghanta lag jayega. Agar wohi kitaben **alphabetical order** mein shelf par hon to seconds mein mil jayegi. Kitabon ko tarteeb se rakhna = **data structure**. Dhoondne ka tareeqa (beech se khol kar aage/peeche jana) = **algorithm**.

Rozmarra zindagi mein DSA

Cheez jo tum use karte ho	Andar kaunsa DSA hai	Kaise?
Undo / Redo (Word, Photoshop)	**Stack** (do stacks)	Har action undo-stack mein; undo karo to woh redo-stack mein chala jata hai
Google Maps	**Graph**	Jagahen = nodes, sarkein = edges; shortest path algorithm
Computer ke folders	**Tree**	Folder ke andar folder — parent/child
Browser back/forward	Stack / Linked list	Pichla page, agla page
Music playlist "next song"	Linked list	Har gaana agle gaane ki taraf ishara karta hai
Contacts mein naam dhoondna	Sorted array + Binary search	Har baar aadha hissa chhod do

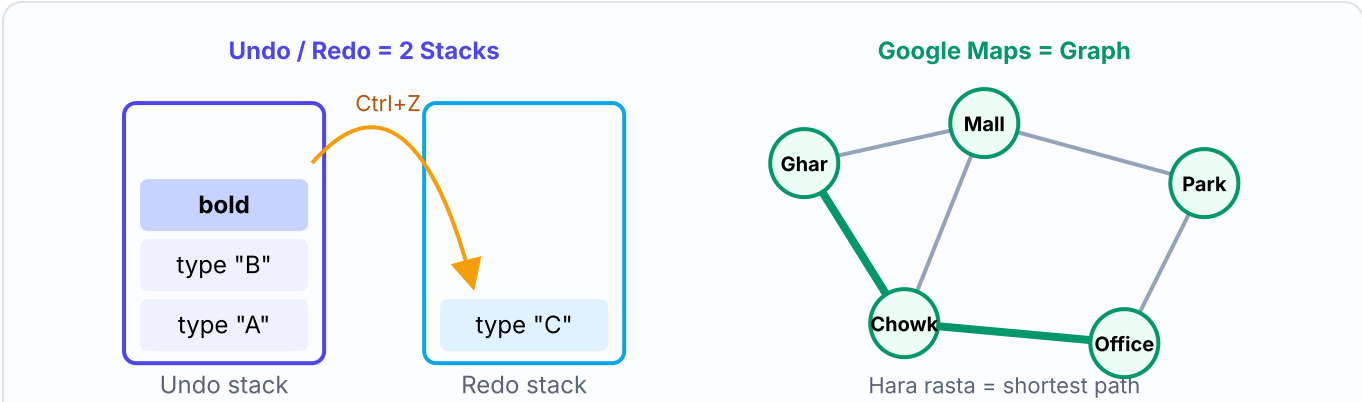


Diagram 1.1 — Undo/Redo do stacks se aur Google Maps graph se chalta hai

DSA kyun seekhein?

- **Interviews:** bari companies masle hal karne ki salahiyat DSA se check karti hain.
- **Behtar code:** sahi data structure chunne se program 1000 guna tez ho sakta hai.
- **Sochne ka tareeqa:** bara masla chhote hisson mein todna — yehi asal skill hai.
- **AI bhi DSA hai:** search, graphs, trees, matrices — sab ke peeche yehi concepts hain.

> **TIP — PYTHON KAAFI HAI**
> Is series mein sab code Python mein hai. Functions, scope aur slicing (`l[1:]` , `s[::-1]`) aani chahiye — baaqi sab yahan seekh loge.

CHAPTER 02

Recursion ki Bunyaad

Jab function khud ko hi call kare

Recursion ka matlab: ek function apne aap ko call karta hai — lekin har baar **chhote input** ke saath — jab tak ek aisi halat na aa jaye jahan jawab seedha pata ho (**base case**).

MISAL: CINEMA KI LINE

Andhere cinema mein tum jaanna chahte ho ke tum kaunsi row mein ho. Tum aagay wale se poochte ho "tum kaunsi row mein ho?" Woh apne aagay wale se poochta hai... sab se aagay wala kehta hai "main row 1 mein hoon" (**base case**). Phir jawab wapas aata hai: 1 → 2 → 3 ... Har banda sirf **ek kaam** karta hai: jo jawab mila us mein +1.

Factorial — pehla recursive function

`n! = n × (n-1)!` aur `0! = 1`. Dekho, factorial ki definition hi recursive hai.

```
def factorial(n):
    if n == 0:                # base case
        return 1
    small_answer = factorial(n - 1)   # chhota masla (trust karo)
    return n * small_answer        # apna kaam

print(factorial(5))   # 120
```

Call Stack — andar kya hota hai?

Har function call **call stack** par ek "plate" (frame) rakhta hai. Jab function khatam hota hai, plate utha li jati hai (LIFO — Last In First Out). Recursion mein plates **neeche se oopar** lagti hain aur jawab **oopar se neeche** wapas aata hai.



Diagram 2.1 — factorial(3) ka call stack: pehle sab calls jama hoti hain, phir ek ek kar ke jawab wapas aata hai

Base case kyun zaroori hai?

GALTI MAT KARNA — BASE CASE BHOOL JANA

Agar base case nahi hai to function kabhi nahi rukega. Har call memory mein plate rakhti jayegi aur aakhir mein `RecursionError: maximum recursion depth exceeded` aa jayega. Ye **stack overflow** hai.

- Python ki default recursion limit taqreeban **1000** hai (`sys.getrecursionlimit()`).
- Isay barhaya ja sakta hai: `sys.setrecursionlimit(10**6)` — lekin bina soche mat barhao, asal masla aksar ghalat base case hota hai.
- Base case aisa ho ke **har** raasta us tak pohanchay. Masalan `factorial(-1)` ke liye `n == 0` kabhi nahi aayega — is liye `n <= 0` zyada mehfooz hai.

YAAD RAKHO

- Recursion = function khud ko **chhote input** ke saath call kare.
- Har recursive function mein: **base case** + **recursive call** + **apna kaam**.
- Call stack LIFO hai — aakhri call sab se pehle khatam hoti hai.
- Recursion muft nahi — har call memory leti hai (Chapter 10 dekho).

CHAPTER 03

PMI — Recursion ka Recipe

Principle of Mathematical Induction: recursion sahi kyun kaam karta hai

Sab se bara sawal: *"Function abhi likha hi ja raha hai, to main* `factorial(n-1)` *par bharosa kyun karun?"* Iska jawab maths ka **PMI** deta hai.

MISAL: DOMINOES

Ek lakh dominoes line mein khare hain. Saabit karna hai ke **sab** girenge. Har ek check karne ki zaroorat nahi — sirf do baatein saabit karo: **(1)** pehla domino girta hai, **(2)** agar koi bhi domino gire to agla bhi girega. Bas — sab gir jayenge!



Diagram 3.1 — PMI = dominoes: base case + "agar k sahi to k+1 sahi"

PMI ka step	Maths mein	Recursion mein
1. Base case	n = 1 (ya 0) par saabit karo	Sab se chhota input jiska jawab seedha pata hai
2. Assumption	Maan lo n = k par sahi hai	Maan lo `f(n-1)` sahi jawab deta hai — **sawal mat karo**
3. Induction step	Saabit karo k+1 par bhi sahi	Chhote jawab se bara jawab banao (apna kaam)

Recursion Recipe — har sawal par ye 3 steps

1. **Base case likho:** sab se chhota input kya hai aur uska jawab kya hai?
2. **Chhote masle par bharosa karo:** `small = f(chhota_input)` — andar mat jhaanko, maan lo sahi aayega.
3. **Apna kaam karo:** `small` ko use kar ke poore input ka jawab banao.

Recipe se hal kiye gaye sawal

Sum of first n numbers

```
def sum_n(n):
    if n == 0: return 0
    return n + sum_n(n - 1)
```

Power of 2 (2^n)

```
def power2(n):
    if n == 0: return 1
    small = power2(n - 1)
    return 2 * small
```

Number of digits

```
def count_digits(n):
    if n < 10: return 1    # 1 digit
    return 1 + count_digits(n // 10)

count_digits(4521)  # 4
```

GCD (Euclid)

```
def gcd(a, b):
    if b == 0: return a
    return gcd(b, a % b)
```

TIP — "TRUST KARO, ANDAR MAT JAO"

Recursion likhte waqt poora tree dimagh mein chalane ki koshish mat karo. Sirf ye socho: "Agar `f(n-1)` mujhe sahi jawab de de, to main `f(n)` kaise banaun?" Bas. Tree banana sirf **samajhne** aur **debugging** ke liye hai, likhne ke liye nahi.

YAAD RAKHO

Base case → Assume → Apna kaam. Ye teen line ka formula 90% recursion sawal hal kar deta hai.

CHAPTER 04

Fibonacci, Recursion Tree, Head vs Tail

Jab ek function do baar khud ko call kare

Fibonacci

Series: `0, 1, 1, 2, 3, 5, 8, 13 ...` — har number pichle do ka jama. `fib(n) = fib(n-1) + fib(n-2)`, base: `fib(0)=0`, `fib(1)=1`.

```
def fib(n):
    if n == 0 or n == 1:     # do base cases — kyun ke do peeche ki calls hain
        return n
    return fib(n - 1) + fib(n - 2)
```

GALTI MAT KARNA

Agar sirf `n == 0` base case rakha to `fib(1)` → `fib(0) + fib(-1)` → `fib(-1)` kabhi nahi rukega. Jitni peeche ki calls hain, utne base cases socho.

Recursion Tree — fib(4)

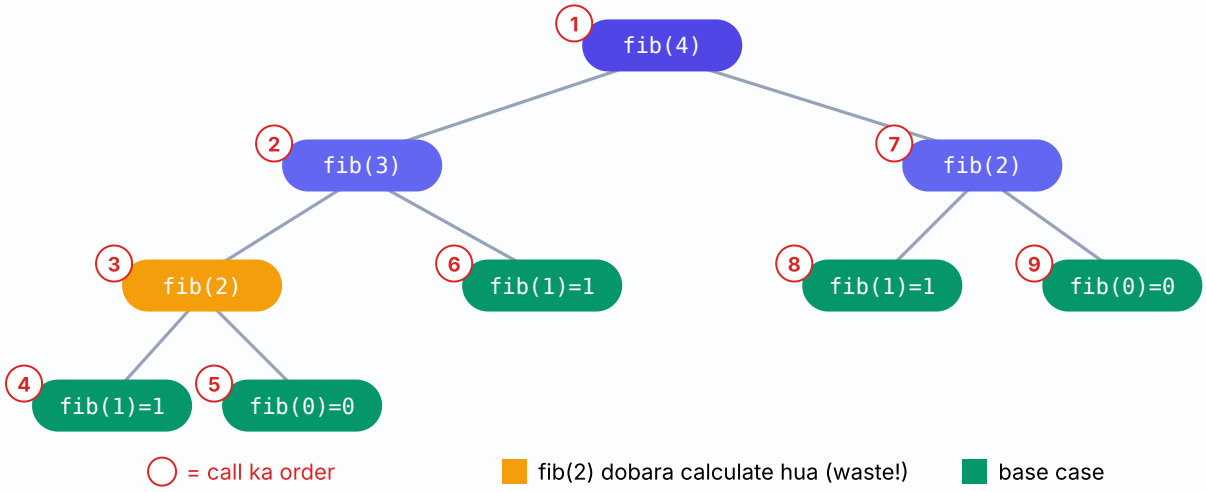


Diagram 4.1 — fib(4) ka recursion tree. Laal number batate hain ke calls kis tarteeb mein hoti hain: pehle bayan (left) taraf poora neeche, phir dayan (right)

- **Call order:** Python pehle `fib(n-1)` poora khatam karta hai (neeche tak), tab `fib(n-2)` shuru hota hai. Yani depth-first.
- **Jawab:** fib(4) = fib(3) + fib(2) = 2 + 1 = **3**.
- **Masla:** wohi kaam baar baar ho raha hai (fib(2) do baar). n bara ho to calls taqreeban **2^n** ho jati hain — bohat slow. (Iska ilaaj aagay Dynamic Programming mein hai.)

1 se n tak aur n se 1 tak print karna

```python
def print_n_to_1(n):
    if n == 0: return
    print(n)              # PEHLE kaam
    print_n_to_1(n - 1)  # PHIR call
# 5 4 3 2 1
```

```python
def print_1_to_n(n):
    if n == 0: return
    print_1_to_n(n - 1)  # PEHLE call
    print(n)              # PHIR kaam
# 1 2 3 4 5
```

Sirf **do lines ki jagah badalne** se output ulta ho gaya! Ye hi head aur tail recursion ka farq hai.

Head vs Tail Recursion

	Tail Recursion	Head Recursion
Kaam kab?	Pehle apna kaam, recursive call **aakhir** mein	Pehle recursive call, apna kaam **wapsi** par
Order	Jaate waqt kaam (oopar se neeche)	Laut-te waqt kaam (neeche se oopar)
Misal	print n → 1	print 1 → n

MISAL: COPY CHECK KARNA

Tail: Teacher pehli copy check karti hai, phir agli copy agle teacher ko de deti hai — kaam jaate hue ho raha hai.

Head: Teacher pehle saari copies neeche wale ko de deti hai; jab sab wapas aati hain tab apni copy check karti hai — kaam lautte hue.

Helper Functions

Kabhi user sirf `n` deta hai, lekin recursion ko extra cheez chahiye (jaise counter `i`). Tab ek **helper** banao jo extra parameter le, aur main function usay call kare.

```python
def print_1_to_n(n):
    helper(1, n)          # user ko sirf n dena hai

def helper(i, n):
    if i > n: return
    print(i)
    helper(i + 1, n)
```

YAAD RAKHO

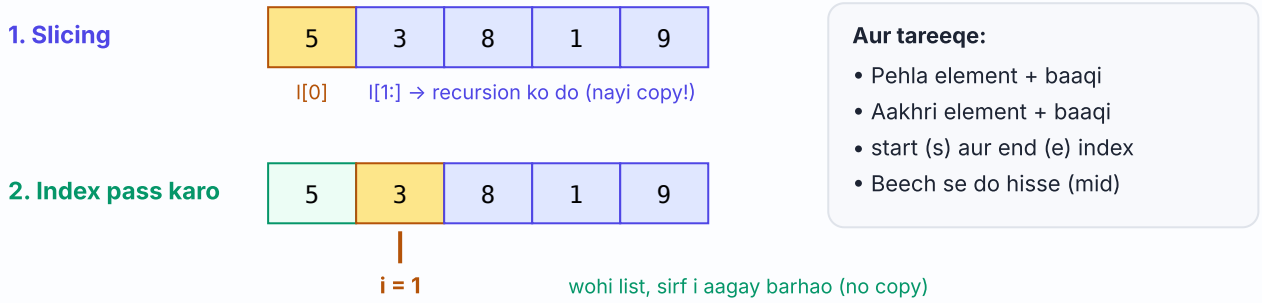
- Jitni peeche ki calls (n-1, n-2), utne base cases.
- Print/kaam ki jagah (call se pehle ya baad) output ka order tay karti hai.
- Extra parameter chahiye? Helper function banao — default arguments (`i=0`) bhi chal jate hain.

CHAPTER 05

Arrays par Recursion

Array ko chhota kaise karein — aur bara sawal kaise hal karein

Array ko todne ke tareeqe



Aur tareeqe:

- Pehla element + baaqi
- Aakhri element + baaqi
- start (s) aur end (e) index
- Beech se do hisse (mid)

Diagram 5.1 — Slicing har call par nayi list banata hai (memory + time); index pass karna sasta hai

Sum of array

```
def arr_sum(l):
    if len(l) == 0: return 0
    return l[0] + arr_sum(l[1:])

def arr_sum_idx(l, i=0):        # behtar: koi copy nahi
    if i == len(l): return 0
    return l[i] + arr_sum_idx(l, i + 1)
```

Kya array sorted hai?

Version 1 — pehle call, phir check

```
def is_sorted(l):
    if len(l) <= 1: return True
    small = is_sorted(l[1:])
    if not small: return False
    return l[0] <= l[1]
```

Version 2 — pehle check (behtar)

```
def is_sorted2(l):
    if len(l) <= 1: return True
    if l[0] > l[1]: return False
    return is_sorted2(l[1:])
```

TIP

Version 2 pehli ghalti milte hi ruk jata hai — `[9, 1, 2, 3 ... 1000 items]` par sirf 1 comparison! Version 1 poori list ke neeche tak jata hai phir check karta hai. Jahan jaldi faisla ho sakta ho, pehle check karo.

First index, Last index, All indices

```
def first_index(l, x, i=0):
    if i == len(l): return -1          # nahi mila
```

```
    if l[i] == x: return i              # aagay se dekho → pehla
    return first_index(l, x, i + 1)

def last_index(l, x, i=0):
    if i == len(l): return -1
    small = last_index(l, x, i + 1)        # pehle peeche wala hissa
    if small != -1: return small        # peeche mil gaya → wohi aakhri
    return i if l[i] == x else -1
```

All indices — 3 tareeqe

Tareeqa	Kaise	Kab use karein
1. Print karo	Mile to `print(i)`, phir aagay call	Sirf dikhana ho
2. List pass karo	Ek hi list sab calls mein jati hai; `ans.append(i)`	Simple aur memory-friendly
3. Nayi list return karo	Chhoti call ki list lo, agar match hai to `[i] + small`	"Pure" recursion — PMI style

```
def all_idx_pass(l, x, i, ans):        # tareeqa 2
    if i == len(l): return
    if l[i] == x: ans.append(i)
    all_idx_pass(l, x, i + 1, ans)

def all_idx_ret(l, x, i=0):            # tareeqa 3
    if i == len(l): return []
    small = all_idx_ret(l, x, i + 1)
    return [i] + small if l[i] == x else small
```

YAAD RAKHO

Slicing (`l[1:]`) har call par O(n) copy banata hai. Interview mein **index pass karo** — wohi kaam, kam memory aur tez.

CHAPTER 06

Searching & Sorting (Recursion se)

Linear search, Binary search, Merge sort, Quick sort

Linear Search

Ek ek element check karo jab tak mil na jaye. Sorted ho ya na ho, chalta hai. Time: **O(n)**.

```
def linear_search(l, x, i=0):
    if i == len(l): return -1
    if l[i] == x: return i
    return linear_search(l, x, i + 1)
```

Binary Search — sirf sorted array par

MISAL: DICTIONARY

Lughat (dictionary) mein "Mango" dhoondna ho to tum pehle page se shuru nahi karte. Beech se kholte ho; agar "P" aa gaya to aagay ka aadha hissa chhod do. Har qadam par aadha kaam khatam!

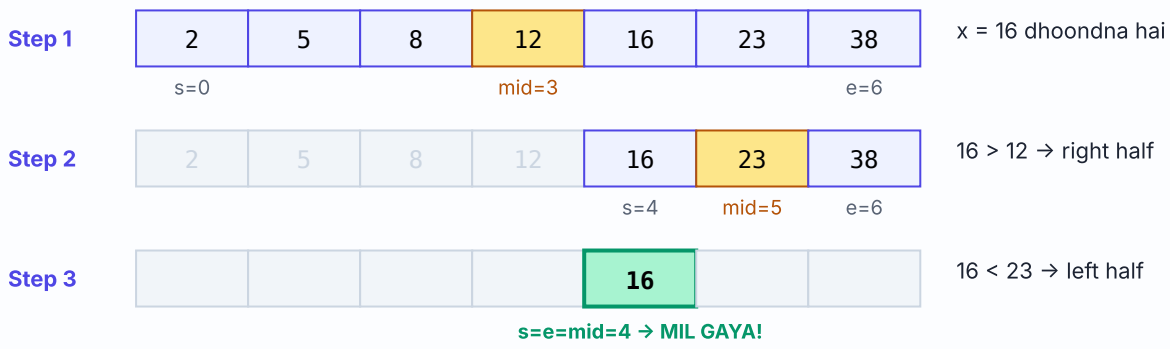


Diagram 6.1 — Binary search: 7 elements mein sirf 3 qadam. 10 lakh elements mein sirf ~20 qadam!

```
def binary_search(l, x, s, e):
    if s > e: return -1                 # range khali → nahi mila
    mid = s + (e - s) // 2                # (s+e)//2 bhi chalta hai
    if l[mid] == x: return mid
    if x < l[mid]: return binary_search(l, x, s, mid - 1)
    return binary_search(l, x, mid + 1, e)
```

TIP — MID = S + (E - S) // 2 KYUN?

C++/Java mein `s + e` bohat bara ho kar **overflow** kar sakta hai. `s + (e-s)//2` wohi jawab deta hai bina overflow ke. Python mein overflow nahi hota, lekin aadat achhi hai.

Merge Sort — "todo aur jodo" (Divide & Conquer)

1. **Divide:** array ko beech se do hisson mein todo.
2. **Trust (PMI):** dono hisson ko recursion se sort karwa lo.
3. **Merge:** do sorted hisson ko jod kar ek sorted array banao.

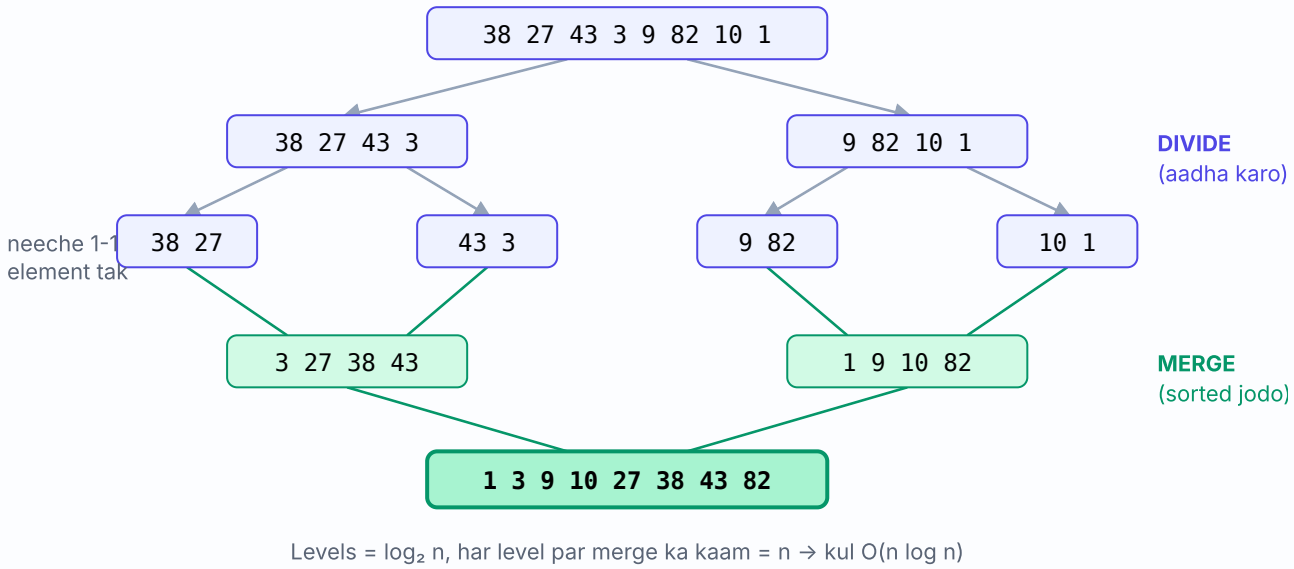


Diagram 6.2 — Merge sort: oopar se neeche todo (blue), neeche se oopar jodo (green)

```
def merge(l, s, m, e):
    i, j, temp = s, m + 1, []
    while i <= m and j <= e:          # dono hisson ke chhote ko pehle uthao
        if l[i] <= l[j]: temp.append(l[i]); i += 1
        else:            temp.append(l[j]); j += 1
    temp.extend(l[i:m + 1])               # bacha hua left
    temp.extend(l[j:e + 1])               # bacha hua right
    l[s:e + 1] = temp                     # wapas asal list mein

def merge_sort(l, s, e):
    if s >= e: return                   # 0 ya 1 element = already sorted
    m = s + (e - s) // 2
    merge_sort(l, s, m)
    merge_sort(l, m + 1, e)
    merge(l, s, m, e)
```

DEBUGGING TIP

Merge sort ghalat aaye to pehle **sirf `merge` ko alag test karo**: `l=[1,4,7,2,3,9]; merge(l,0,2,5)`. Agar merge sahi hai to recursion khud sahi hoga (PMI!).

Quick Sort — pivot ko uski sahi jagah bhejo

1. **Pivot chuno:** yahan aakhri element `l[e]`.

2. **Count karo:** kitne elements pivot se chhote hain → `c`. Pivot ki sahi jagah index `s + count` hai.

3. **Pivot ko wahan swap karo.**

4. **Partition:** `i` left se, `j` right se — left mein bara mile aur right mein chhota mile to swap.

5. **Recursion:** pivot ke left aur right hisse ko alag sort karo.

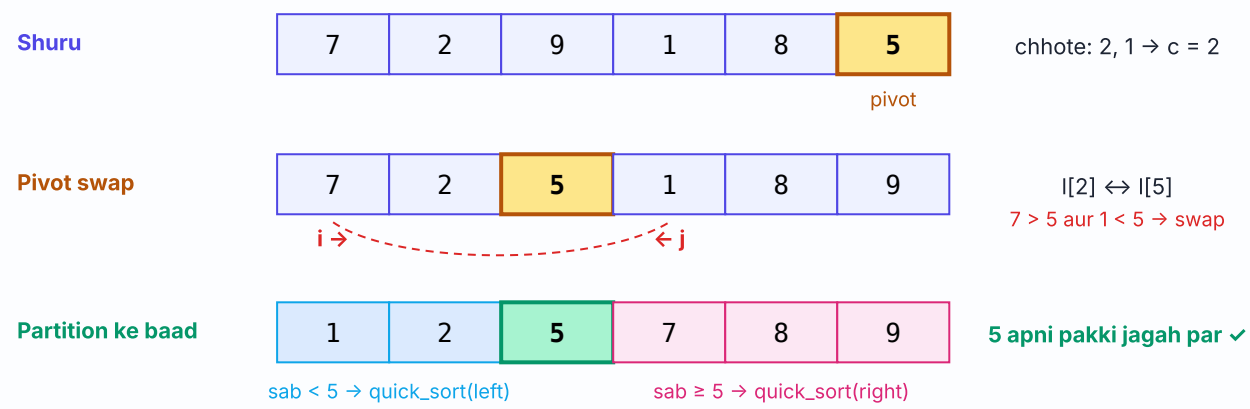


Diagram 6.3 — Quick sort ka partition: pivot ko sahi index par rakh kar dono taraf ko alag sort karo

```
def partition(l, s, e):
    pivot = l[e]
    c = s
    for i in range(s, e):              # kitne chhote hain?
        if l[i] < pivot: c += 1
    l[c], l[e] = l[e], l[c]               # pivot sahi jagah par
    i, j = s, e
    while i < c and j > c:
        if l[i] < pivot:    i += 1       # left mein theek hai
        elif l[j] >= pivot: j -= 1       # right mein theek hai
        else:
            l[i], l[j] = l[j], l[i]; i += 1; j -= 1
    return c

def quick_sort(l, s, e):
    if s >= e: return
    p = partition(l, s, e)
    quick_sort(l, s, p - 1)
    quick_sort(l, p + 1, e)
```

GALTI MAT KARNA — BARABAR ELEMENTS (TIES)

Pivot ke barabar wale element kis taraf jayenge, ye **ek jaisa** hona chahiye. Count mein `< pivot` liya, to partition mein bhi left = `<` aur right = `>=`. Mix kiya to duplicates par infinite loop ya ghalat jawab aayega.

	Merge Sort	Quick Sort
Asal kaam kahan	Wapsi par (merge)	Jaate waqt (partition)
Time	Hamesha O(n log n)	Average O(n log n), worst $O(n^2)$ (sorted array + last pivot)
Extra space	O(n) temp list	In-place (sirf recursion stack)

CHAPTER 07

Strings, Subsequence & Permutations

String bhi characters ki list hai — wohi recipe chalti hai

String se 'z' hatao

```
def remove_z(s):
    if len(s) == 0: return ""
    small = remove_z(s[1:])
    return small if s[0] == 'z' else s[0] + small

remove_z("pizza")   # "pia"
```

Palindrome check (s aur e pointers)

Palindrome = aagay aur peeche se ek jaisa (`"madam"`, `"racecar"`). Dono kinaron ko milao aur andar ki taraf barho.

```
def is_pal(s, st, e):
    if st >= e: return True           # beech mein pohanch gaye
    if s[st] != s[e]: return False
    return is_pal(s, st + 1, e - 1)

# LeetCode "Valid Palindrome": pehle saaf karo
text = "A man, a plan, a canal: Panama"
clean = "".join(ch.lower() for ch in text if ch.isalnum())
is_pal(clean, 0, len(clean) - 1)    # True
```

Substring vs Subsequence

SUBSTRING = HOSTEL KI MESS LINE

Line mein jo jaisa khara hai, waisa hi lena hai — **beech se koi skip nahi**. Lagataar (contiguous) hissa. `"abc"` ke substrings: a, b, c, ab, bc, abc.

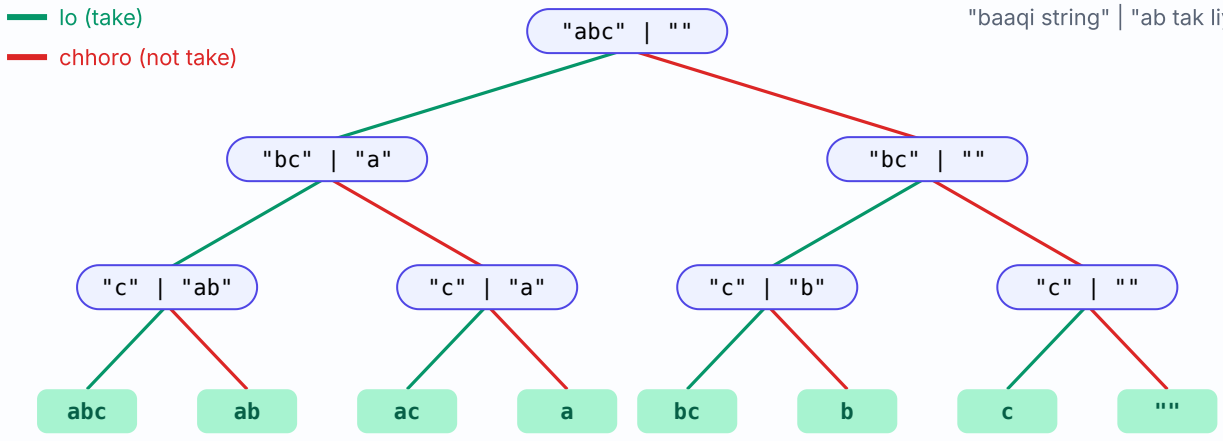
SUBSEQUENCE = SHAADI KA BUFFET

Order wohi rehta hai (pehle starter, phir main course), lekin **koi bhi dish skip** kar sakte ho. `"abc"` → "", a, b, c, ab, ac, bc, abc. **"ac"** subsequence hai, substring nahi.

GINTI

n characters → substrings = **n(n+1)/2** (khali ke baghair), subsequences = **2^n** (khali ke saath) — kyun ke har character ke paas 2 choice hai: **lo ya chhoro**.

Tareeqa 1: Take / Not-take (choices tree)



Har level par 2 raaste → 3 characters = 2^3 = 8 subsequences (leaves)

Diagram 7.1 — Subsequences ka choices tree: har character par "lo ya chhoro"

```
def subseq(s, out=""):
    if len(s) == 0:
        print(out); return
    subseq(s[1:], out + s[0])    # lo
    subseq(s[1:], out)            # chhoro
```

Tareeqa 2: Chhote jawab ke har subset mein jodo (return list)

PMI style: `"bc"` ke subsequences maan lo mil gaye → `["", "b", "c", "bc"]`. Ab `"abc"` ke liye: yehi 4 (bina 'a') + har ek ke aagay 'a' laga do (4 aur) = 8.

```
def subseq_list(s):
    if len(s) == 0: return [""]
    small = subseq_list(s[1:])
    return small + [s[0] + x for x in small]
```

Permutations — saari tarteebein

`"abc"` ki permutations: abc, acb, bac, bca, cab, cba — kul **n! = 6**. Subsequence mein kuch chhod sakte the; yahan **sab** use karne hain, sirf order badalna hai.

IDEA: NAYA CHARACTER HAR KHAALI JAGAH MEIN GHUSAO

`"bc"` ki permutations maan lo: `bc, cb`. Ab 'a' ko har jagah rakho:
`_b_c_` → a**b**c, ba**c**, bc**a** | `_c_b_` → a**c**b, c**a**b, cb**a**. Kul 6 ✓

```
def perms(s):
    if len(s) <= 1: return [s]
    small = perms(s[1:])
    ans = []
    for p in small:
        for k in range(len(p) + 1):        # har slot: 0..len(p)
            ans.append(p[:k] + s[0] + p[k:])  # slicing se ghusao
    return ans
```

DUSRA TAREEQA (CHOICE TREE)

Har qadam par bachi hui string mein se **koi ek** character chuno, usay output mein jodo, baaqi par recursion: pehle level par n choices, phir n-1, phir n-2 ... isi liye kul n! leaves.

CHAPTER 08

Keypad Combinations & Tower of Hanoi

Do mashhoor recursion sawal

Letter Combinations of a Phone Number (LeetCode 17)

Purane mobile keypad par har digit ke saath letters hote hain. Digits `"23"` diye hon to saare mumkin words batao.

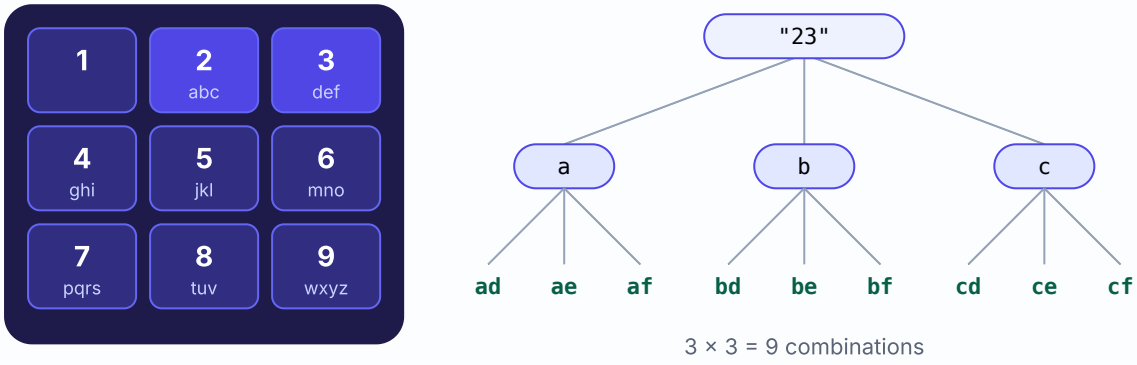


Diagram 8.1 — "23": pehle digit ke har letter ke saath doosre digit ka har letter

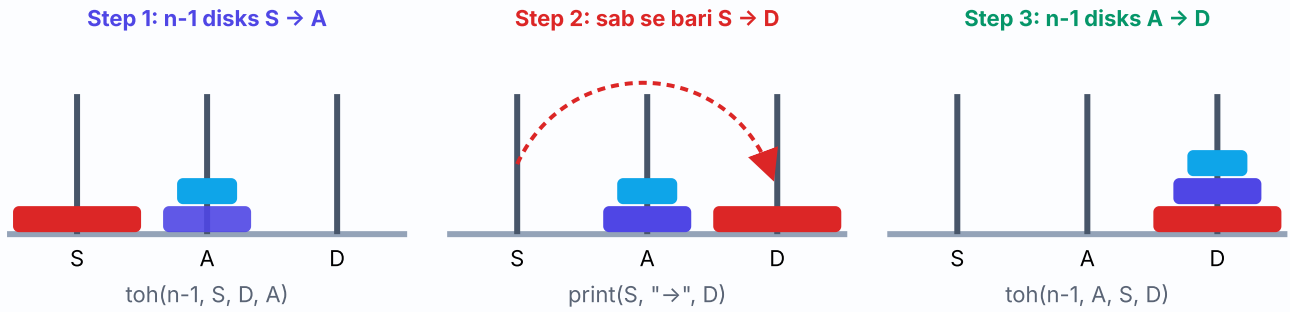
```
KEYS = {'2':'abc', '3':'def', '4':'ghi', '5':'jkl',
        '6':'mno', '7':'pqrs', '8':'tuv', '9':'wxyz'}

def letter_combos(digits):
    if len(digits) == 0: return [""]
    small = letter_combos(digits[1:])          # baaqi digits ke words
    return [ch + w for ch in KEYS[digits[0]] for w in small]

# LeetCode: agar digits khali ho to [] return karna hai — wo case alag handle karo
```

Tower of Hanoi

Teen rods hain: **Source (S)**, **Auxiliary (A)**, **Destination (D)**. n disks S par hain (bari neeche). Sab ko D par le jao. Qawaid: **(1)** ek waqt mein ek disk, **(2)** bari disk kabhi chhoti ke oopar nahi.



Bari disk ke liye chhoti n-1 disks ka "pahaar" hatao, bari ko bhejo, pahaar wapas rakh do.

Moves: $T(n) = 2 \cdot T(n-1) + 1 \rightarrow 2^n - 1$ (3 disks = 7 moves, 64 disks = ~1.8×10^{19} moves!)

Diagram 8.2 — Tower of Hanoi ke 3 qadam (PMI: n-1 disks wala kaam recursion par chhor do)

```
def toh(n, s, a, d):
    if n == 0: return
    toh(n - 1, s, d, a)        # n-1 ko S se A par (D madadgaar)
    print(s, '->', d)             # sab se bari disk
    toh(n - 1, a, s, d)        # n-1 ko A se D par (S madadgaar)

toh(3, 'S', 'A', 'D')
# S->D  S->A  D->A  S->D  A->S  A->D  S->D   (7 moves)
```

YAAD RAKHO

Tower of Hanoi PMI ki sab se khoobsurat misal hai: tum sirf **ek** disk (sab se bari) khud hilate ho; baaqi n-1 ka kaam "trust" kar ke recursion ko de dete ho — sirf rods ke naam badal kar.

CHAPTER 09

Time Complexity

Code kitna tez hai — input barhne par kaam kitna barhta hai

Time taken ≠ Time complexity

Ek hi code purane laptop par 5 second aur naye par 1 second le sakta hai. To "kitne second laga" se algorithm ki quality pata nahi chalti. **Time complexity** ye batati hai ke **input (n) barhne par operations kis raftaar se barhte hain**.

EXPERIMENTAL ANALYSIS (STOPWATCH SE NAAPNA) KE MASLE

- Machine par depend karta hai (CPU, RAM, background apps).
- Har input ke liye chalana parta hai — kaunse inputs? Worst case shayad test hi na ho.
- Do algorithms compare karne ke liye dono ko **likhna aur chalana** parta hai — waqt zaya.

Is liye hum **operations ginte** hain (har basic kaam = 1 unit) aur dekhte hain ke ye n ke saath kaise barhte hain.

Asymptotic Notations

Notation	Matlab	Misal (asaan zaban)
O (Big-O)	Upper bound — **is se zyada nahi** (worst case)	"Ghar pohanchne mein zyada se zyada 1 ghanta"
Ω (Omega)	Lower bound — **is se kam nahi** (best case)	"Kam se kam 20 minute to lagenge"
Θ (Theta)	Tight bound — oopar aur neeche dono yehi	"Taqreeban theek 40 minute"
o (little-o)	Strict upper bound (barabar nahi, sakhti se kam)	$n = o(n^2)$, lekin $n \neq o(n)$
ω (little-omega)	Strict lower bound	$n^2 = \omega(n)$

INTERVIEW MEIN

Jab koi "complexity batao" kahe to aam taur par **worst case Big-O** maanga ja raha hota hai.

Big-O nikalne ke 2 qanoon

1. CHHOTE TERMS HATA DO

`n² + 100n + 5000` → **O(n²)**. n = 10^6 par n^2 = 10^{12} aur 100n = 10^8 — chhota term kuch maayne nahi rakhta.

2. CONSTANTS HATA DO

`5n` → **O(n)**, `n²/2` → **O(n²)**. Constant machine jaisa hai — growth ki shakal nahi badalta.

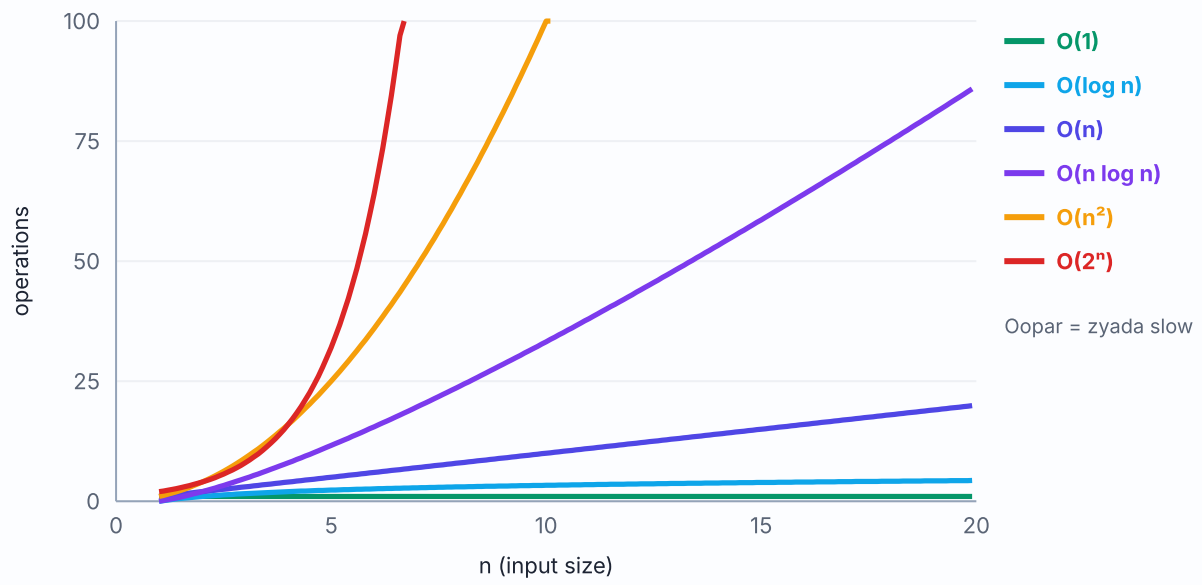


Diagram 9.1 — Mukhtalif complexities ki growth: n barhne par O(2ⁿ) aur O(n²) kitni tezi se oopar jati hain

Aam misalen

Code	Complexity	Kyun
Array ka beech wala element `l[n//2]`	O(1)	n kitna bhi ho, ek hi kaam
Array ka maximum dhoondna	O(n)	Har element ek baar dekhna
Bubble / Insertion / Selection sort	O(n²)	Loop ke andar loop
`i = 1; while i < n: i *= 2`	O(log n)	Har baar double → $\log_2 n$ qadam
`while n > 0: n //= 2`	O(log n)	Har baar aadha
Do alag (nested nahi) loops 0..n	O(n)	n + n = 2n → O(n)

```
# Nested loop jahan andar wala i se shuru — phir bhi O(n²)
for i in range(n):
    for j in range(i, n):    # n + (n-1) + ... + 1 = n(n+1)/2 → O(n²)
        ...
```

Recursion ki complexity — Recurrence Relations

Recursive function ke liye ek equation likho: **T(n) = (chhoti calls ka time) + (apna kaam)**, phir khol kar hal karo.

Algorithm	Recurrence	Hal	Answer
Factorial	$T(n) = T(n-1) + k$	k + k + ... (n baar)	**O(n)**
Binary Search	$T(n) = T(n/2) + k$	n → n/2 → ... → 1: log n qadam	**O(log n)**
Merge Sort	$T(n) = 2T(n/2) + kn$	log n levels × har level n kaam	**O(n log n)**
Fibonacci	$T(n) = T(n-1) + T(n-2) + k$	Har call 2 calls → tree dugna	**$O(2^n)$** (asal mein ≈ φ^n, $\varphi \approx 1.618$)
Tower of Hanoi	$T(n) = 2T(n-1) + k$	$1 + 2 + 4 + ... + 2^{n-1}$	**$O(2^n)$**

FACTORIAL KO KHOL KAR DEKHO

$T(n) = T(n-1) + k$
$= T(n-2) + 2k$
$= T(n-3) + 3k$
$= ... = T(0) + n \cdot k \rightarrow$ **O(n)**

Constraints se complexity ka andaza (bohat kaam ki cheez!)

Online judge (LeetCode, Codeforces) par taqreeban **10^8 operations ≈ 1 second**. Sawal mein di gayi n ki hadd dekh kar pata chal jata hai kaunsi complexity chalegi:

n ki hadd	Chalne wali complexity	Aam algorithm
$n \le 12$	$O(n!)$	Saari permutations
$n \le 25$	$O(2^n)$	Saari subsequences / subsets
$n \le 100$	$O(n^4)$	4 nested loops
$n \le 500$	$O(n^3)$	3 nested loops
$n \le 10^4$	$O(n^2)$	Bubble/insertion sort, 2 loops
$n \le 10^6$	$O(n \log n)$	Merge sort, sorting + kuch
$n \le 10^8$	$O(n)$	Ek loop
$n > 10^8$	$O(\log n)$ ya $O(1)$	Binary search, formula

CHAPTER 10

Space Complexity

Code kitni memory leta hai

Total space = Input space + Auxiliary space. Input to dena hi parta hai, is liye aam taur par hum **auxiliary (extra) space** ki baat karte hain — jo algorithm khud apne kaam ke liye leta hai.

MISAL: PARKING LOT

Din bhar mein 500 gaariyan aayin aur gayin, lekin ek waqt mein zyada se zyada 40 khari thin. Parking lot ko 40 jagah chahiye, 500 nahi. Isi tarah space complexity = **kisi bhi ek waqt par sab se zyada (peak)** memory — kul jama nahi.

Code	Extra space	Wajah
Max dhoondna (ek variable)	O(1)	Sirf `max_val`
Bubble / Selection / Insertion sort	O(1)	In-place — wohi list mein swap
Array copy / naya list banana	O(n)	n naye elements
n × n matrix banana	$O(n^2)$	n^2 cells

Recursion muft nahi hai!

Har recursive call stack par ek frame rakhti hai. Isliye recursion ki space = **stack ki zyada se zyada gehraai (depth)** × har frame ka space.

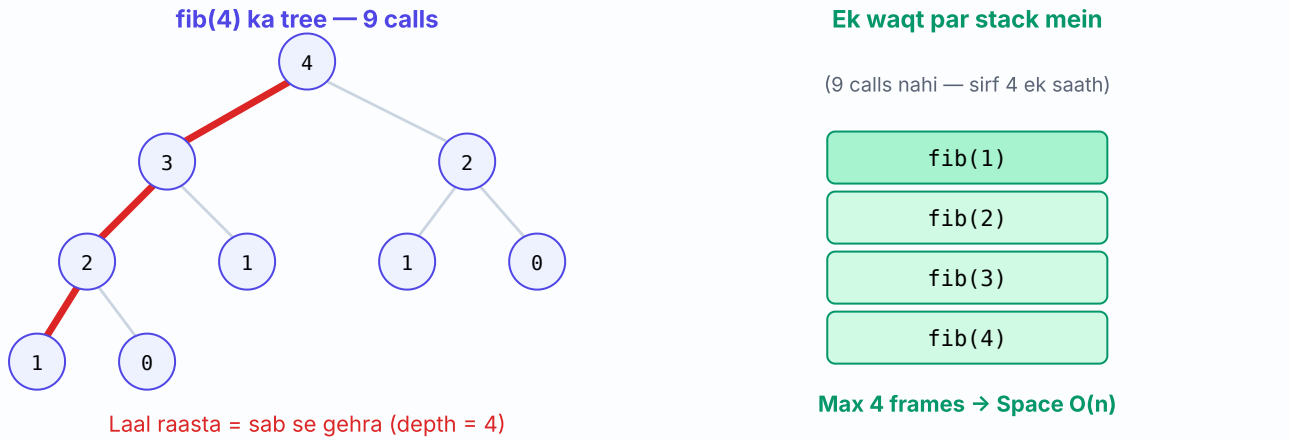


Diagram 10.1 — Fibonacci: time O(2ⁿ) (kul calls) lekin space sirf O(n) (gehraai) — kyun ke calls ek saath stack par nahi hoti

Recursive algorithm	Stack depth	Space
Factorial(n)	n	**O(n)**
Binary search	log n	**O(log n)** (iterative version O(1))
Fibonacci	n	**O(n)** — depth, nodes ki ginti nahi
Merge sort	log n (stack) + n (temp list)	**O(n)**
Quick sort	average log n, worst n	**O(log n)** avg, O(n) worst

GALTI MAT KARNA

"Fibonacci ki 2^n calls hain to space bhi 2^n" — **ghalat!** Jab ek branch khatam hoti hai uska frame stack se hat jata hai. Ek waqt par sirf ek raasta (root se leaf tak) stack par hota hai.

SLICING KA CHHUPA KHARCHA

`f(l[1:])` har call par nayi list banata hai: time mein bhi extra O(n) aur space mein bhi. Index pass karne se ye kharcha khatam ho jata hai.

CHAPTER 11

Linked List

Dabbe jo ek doosre ka pata (address) yaad rakhte hain

Array ke masle — Linked List kyun?

- **Lagataar (contiguous) memory chahiye:** 1000 elements ke liye memory mein ek saath 1000 khaali jagah honi chahiye.
- **Jagah zaya / kam:** pehle se size socho — zyada lo to waste, kam lo to naya bara array banao aur copy karo.
- **Beech mein insert/delete mehnga:** baaqi sab elements ko khiskana (shift) parta hai → O(n).

MISAL: CINEMA SEATS

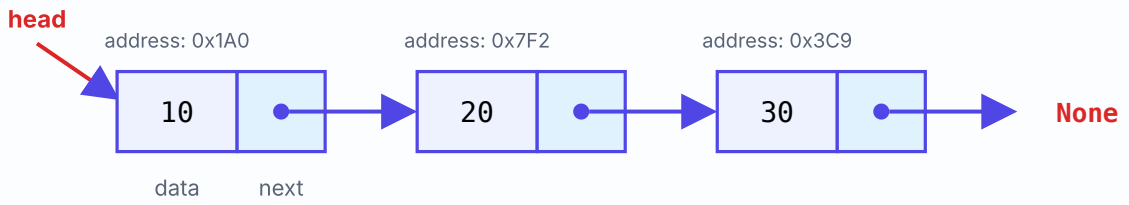
10 doston ko cinema mein **saath wali** 10 seats chahiye (array). Hall mein 30 seats khaali hain lekin bikhri hui — to bhi nahi baith sakte! Linked list kehti hai: jahan jagah mile baith jao, bas har banda agle ki seat number yaad rakhe.

MISAL: TREASURE HUNT / PICNIC

Har parchi par agli parchi ka pata likha hai — pehli parchi (head) mil jaye to aakhir tak pohanch jaoge. Picnic par har bacha sirf apne aagay wale ka haath pakarta hai.

Aur misalen: **music playlist** (next song), **browser history** (pichla/agla page), image viewer (next/previous).

Node — linked list ki eent



Nodes memory mein kahin bhi ho sakte hain — **next** unhe jorta hai. Hum sirf **head** yaad rakhte hain.

Diagram 11.1 — Har node mein do cheezen: data aur agle node ka reference (next). Aakhri node ka next = None

```
class Node:
    def __init__(self, value):
        self.data = value
        self.next = None

def print_ll(head):
    temp = head                    # head ko kabhi mat hilao!
    while temp is not None:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

DO SAB SE BARI GHALTIYAN

- **head ko seedha aagay barhana:** `head = head.next` kiya to shuru ke nodes hamesha ke liye kho gaye. Hamesha `temp` pointer use karo.
- **None par `.next` :** `temp.next` likhne se pehle pakka karo ke `temp` None nahi — warna `AttributeError: 'NoneType' object has no attribute 'next'`.

Input lena — tail pointer ke saath O(n)

```
def take_input(values):
    head = tail = None
    for data in values:
        n = Node(data)
        if head is None:
            head = tail = n           # pehla node
        else:
            tail.next = n             # aakhri ke peeche jodo
            tail = n                  # tail aagay
    return head
```

TIP

Agar har naye node ke liye head se aakhir tak chalo to kul $O(n^2)$ lagega. **tail** yaad rakhne se har insert O(1) → poora input O(n).

Length

```
def length(head):          # iterative
    c, temp = 0, head
    while temp:
        c += 1; temp = temp.next
    return c
```

```
def length_rec(head):      # recursive
    if head is None: return 0
    return 1 + length_rec(head.next)
```

Insertion

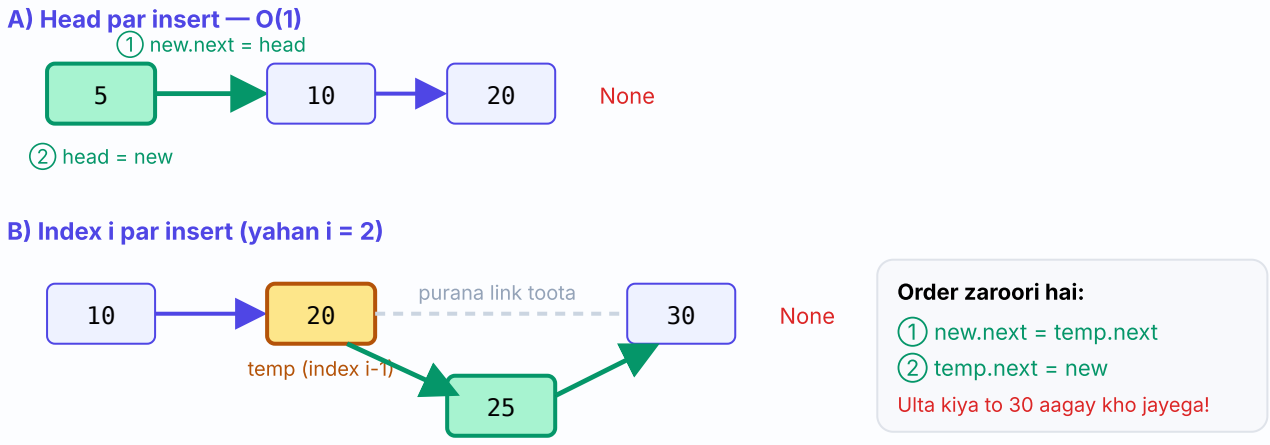


Diagram 11.2 — Head aur beech mein insertion: pehle naye node ko aagay se jodo, phir peeche wale ko naye se

```python
def insert_at_head(head, data):            # O(1)
    new = Node(data)
    new.next = head
    return new                              # naya head

def insert_at_tail(head, data):            # O(n) (tail ho to O(1))
    new = Node(data)
    if head is None: return new
    temp = head
    while temp.next is not None: temp = temp.next
    temp.next = new
    return head

def insert_at_index(head, i, data):
    if i == 0: return insert_at_head(head, data)
    temp, c = head, 0
    while temp is not None and c < i - 1:      # i-1 par ruko
        temp = temp.next; c += 1
    if temp is None: return head               # index list se bahar
    new = Node(data)
    new.next = temp.next                          # ①
    temp.next = new                               # ②
    return head

def insert_rec(head, i, data):              # recursive version
    if i == 0:
        new = Node(data); new.next = head; return new
    if head is None: return None
    head.next = insert_rec(head.next, i - 1, data)
    return head
```

Deletion

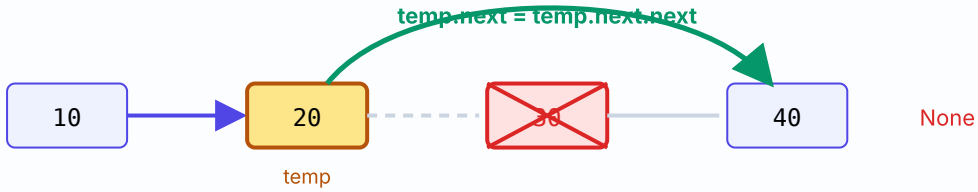


Diagram 11.3 — Delete karna = us node ko "bypass" kar dena. Python ka garbage collector baaqi kaam kar leta hai

```python
def delete_head(head):
    return None if head is None else head.next

def delete_tail(head):
    if head is None or head.next is None: return None
    temp = head
    while temp.next.next is not None:   # doosre-aakhri par ruko
        temp = temp.next
    temp.next = None
    return head

def delete_index(head, i):
    if head is None: return None
    if i == 0: return head.next
    temp, c = head, 0
```

```
    while temp.next is not None and c < i - 1:
        temp = temp.next; c += 1
    if temp.next is not None:
        temp.next = temp.next.next            # bypass
    return head

def delete_value(head, x):                    # pehla x hatao
    if head is None: return None
    if head.data == x: return head.next
    temp = head
    while temp.next is not None and temp.next.data != x:
        temp = temp.next
    if temp.next is not None:
        temp.next = temp.next.next
    return head

def delete_rec(head, i):                      # recursive
    if head is None: return None
    if i == 0: return head.next
    head.next = delete_rec(head.next, i - 1)
    return head
```

Array vs Linked List

Kaam	Array / Python list	Linked List
i-th element padhna	**O(1)**	O(n) — head se chalna parta hai
Shuru mein insert/delete	O(n) — shift	**O(1)**
Aakhir mein insert	O(1) (amortized)	O(n), tail ho to **O(1)**
Beech mein insert (jagah mil gayi ho)	O(n)	**O(1)** — sirf 2 pointers
Memory	Lagataar block chahiye	Bikhri hui chal jati hai (+ har node par extra next)

CHAPTER 12

Quick Revision Cheat-Sheet

Exam / interview se pehle 5 minute mein

RECURSION RECIPE

1. Base case (sab se chhota input)
2. Chhote masle par trust: `small = f(n-1)`
3. Apna kaam: small se bara jawab

HEAD VS TAIL

Kaam call se **pehle** = tail (jaate hue) → n...1
Kaam call ke **baad** = head (laute hue) → 1...n

SUBSTRING VS SUBSEQUENCE

Substring = mess line (lagataar), $n(n+1)/2$
Subsequence = buffet (skip allowed, order same), 2^n
Permutations = sab use, order badlo, $n!$

SORTING YAAD

Merge: aadha karo → dono sort → merge. $O(n \log n)$, $O(n)$ space.
Quick: pivot sahi jagah → left/right sort. Avg $O(n \log n)$, worst $O(n^2)$.

Complexity table — ek nazar mein

Algorithm / Operation	Time	Extra Space
Factorial / sum recursive	$O(n)$	$O(n)$ stack
Fibonacci (simple recursion)	$O(2^n)$	$O(n)$ stack
Linear search	$O(n)$	$O(1)$ iterative
Binary search (recursive)	$O(\log n)$	$O(\log n)$
Bubble / Selection / Insertion	$O(n^2)$	$O(1)$
Merge sort	$O(n \log n)$	$O(n)$
Quick sort	$O(n \log n)$ avg · $O(n^2)$ worst	$O(\log n)$ avg
All subsequences	$O(2^n \cdot n)$	$O(n)$ depth
All permutations	$O(n! \cdot n)$	$O(n)$ depth
Tower of Hanoi	$O(2^n)$	$O(n)$
Linked list: insert at head	$O(1)$	$O(1)$
Linked list: search / index access	$O(n)$	$O(1)$

Growth ka order (chhote se bare)

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$

TOP 6 GHALTIYAN

1. Base case bhool jana / har raasta base case tak na pohanchna.
2. Fibonacci jaisi 2 calls mein sirf ek base case rakhna.
3. `l[1:]` slicing ko muft samajhna — ye O(n) copy hai.
4. Space ko "kul calls" se ginna — asal mein peak stack depth.
5. Linked list mein head ko seedha hilana, ya None par `.next`.
6. Insert mein pointers ka order ulta: pehle `new.next`, phir `temp.next`.

AAGAY KYA?

Repo (github.com/mayank953/HelloDSA) mein har topic ka code, notebooks aur **checkpoint assignments** hain. Har chapter ke baad assignment khud hal karo, phir solution se milao. Agle topics: Stack, Queue, Hashmap, Trees, BST, Heaps, Graphs, DP.